Introduction

In some fields of law, particularly, the field of litigation, time sensitive obligations must be tracked. Litigation is a specialization in which lawyers handle lawsuits and court cases, and the courts impose strict deadlines. However these deadlines are not always explicit.

For example:

- A court order says "defendant must respond within 30 days"
- A notice says "discovery must be completed by June 15, 2025"
- A scheduling order lists multiple hearing dates and filing deadlines

This is what makes this problem an interesting application of LLMs.

If the deadlines are missed, then there can be material damages to the case, such as automatically losing the case, or malpractice lawsuits. The deadlines can be in PDFs, scanned court orders or emails. Paralegals or docket clerks manually read documents and enter dates into specialized software, but this can be a slow, error prone process. The practice of tracking these court imposed deadlines is called docket control or critical date management.

This document aims to propose a solution to automate the extraction of these key dates from digital media including PDFs, Word documents, scanned documents, and exported emails as text files.

Management of the confirmed events is out of scope for this design.

Technical Architecture

Functional Requirements

- System should support multiple media types including PDFs, scanned court orders (jpg, png), or formatted text (e.g, emails)
- System should extract key deadlines, events, obligations from document
- Out of scope: list all docs from a case, map doc to events

Nonfunctional Requirements

- Accuracy of resulting numbers should be at least 99% accurate[1]
- Storage of events, obligations, deadlines should have high consistency
    - Inconsistent data could lead to material damages to case or business.
- Record retrieval should have low latency, under .2 sec
- Should scale to 1000 documents / day with latency no more than 15 mins

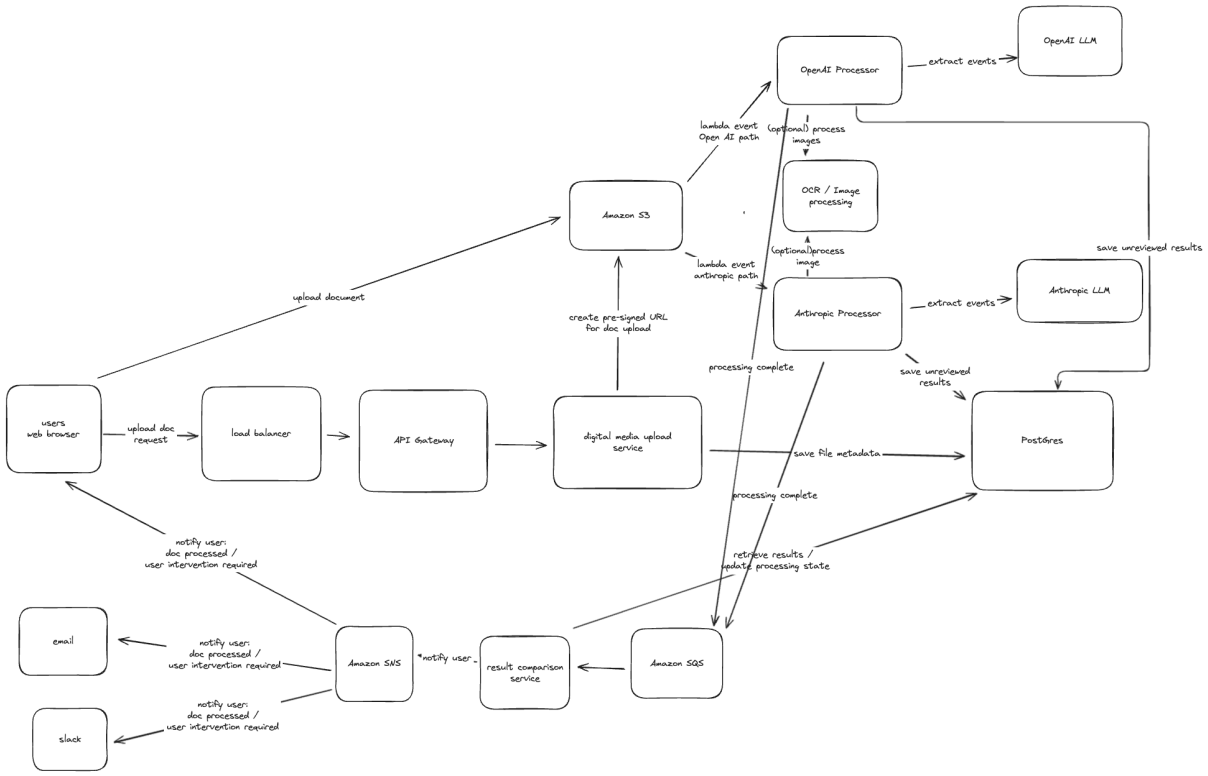
[1] https://www.qualitymag.com/articles/96853-manual-data-entry-and-its-effects-on-quality

Eric Low
eric.low@gmail.com
Nov 11, 2025

Core Entities

Client - A law firm, that is using the system to extract and track events
User - the person that verifies the extracted data, a lawyer or paralegal
Case - the litigation case that is being tracked
Document - an artifact from the litigation case that is expected to have events
ExtractedCalendarEntry - unverified calendar event extracted from doc
CalendarEntry - verified calendar event extracted extracted from doc
Event - a calendar entry could have repeating events, these are real events with date and time

High Level Design



1. Uploading Digital Media
Litigation documentation (PDFs, scanned docs, text files) will be directly uploaded to S3 through a pre-signed URL. File metadata will be saved to Postgres.

2. Processing Digital Media
Once the document is uploaded to S3, AWS will start two lambda processors in parallel, one for OpenAI and one for Anthropic. The processors will retrieve key metadata from the database to assist in text extraction. The processors may employ the assistance of other tooling, such as OCR and image processing. Once the text has been extracted, the text will be saved to the database as an intermediate artifact. This will be useful for reprocessing, and verification, and

Eric Low
eric.low@gmail.com
Nov 11, 2025

auditing. Finally the text will be submitted to the LLM for event and date extraction. Key events, will be saved as ExtractedCalendarEntries in the database.

There are several choices for event extraction, including ML approaches, but ML approaches would need fine tuning to be able to retrieve complex cases such as “within 30 days of service”, also there could be nuances with each court system. LLMs would work out of the box and handle these nuanced cases with little prompt engineering.

In choosing LLMs, it would be preferable to go with Sonnet 4 and GPT-4o, which are the current best in class models.

Once the event extraction is complete, an event will be placed on Amazon SQS to signal that one of the processing steps has been completed.

The consumer of events, the result comparison service, should check the database to see if all the LLM processors have completed their work. Once all the processors are complete, the result comparison should compare the results between the two LLMs. If the two LLM’s agree with high confidence, then we can auto accept or notify the user that event extraction is ready for review. Using two LLMs will reduce systematic error from a single model. There is still a chance that two LLMs make the same mistake, so “human in the loop” for accountability, is a key part of the design.

Extraction confidence: the score should combine LLM agreement and date explicitness. High confidence requires both LLMs to extract identical dates with matching context. Low confidence indicates disagreements or ambiguous language.

3. User Notification

The result comparison service will notify end users by placing a request to Amazon SNS for notification on email, slack, app, or other communication mechanisms. Notification is a key step for human in the loop verification.

Production Considerations

Error handling and edge cases

- OCR Failures: poor image quality, system cannot extract events
    - Handling: flag for manual processing, notify user
- LLM API Failures
    - Handling: retry mechanisms, notify user
- Edge Cases - general strategy: extract information as is, flag the complexity in a confidence score. Attempt to resolve using rules based instruction. Request intervention from user.

Eric Low
eric.low@gmail.com
Nov 11, 2025

- Ex: Relative Dates - “within 30 days of service” - notify user, create specialized UI to provide specialized dates.
- Ex: Amended Orders - an order which modifies a previous deadline.

Scalability considerations

Primary bottleneck: LLM API rate limits

Mitigation - Add a SQS queue, limit the concurrency of lambda processors. If we hit rate limits, retry with exponential backoffs. After 3 attempts, route the message to the dead letter queue, which will retry on a delayed reprocessing schedule.

Trust and compliance mechanisms

All processing steps should generate an audit trail. For example, when verifying the extracted events, date, time, and the person who verified the result should be noted. The final event should have links to the source doc, and intermediate results such as the extracted text.

Eric Low
eric.low@gmail.com
Nov 11, 2025